

Retrieval-Augmented Generation

Yiwen Lu¹ Angeles Marin Batana²

¹Department of Electrical Engineering (EE)
University of Notre Dame

²Department of Computer Science and Engineering (CSE)
University of Notre Dame

CSE 60556 LLM

Table of Contents

- 1 LLM Background
- 2 Before RAG: Where Should Knowledge Live?
- 3 RAG: External Memory Meets Generation
- 4 Why Vanilla RAG Is Not Enough
- 5 After RAG: Weaknesses and Fixes
- 6 Summary and Takeaways

Table of Contents

- 1 LLM Background
- 2 Before RAG: Where Should Knowledge Live?
- 3 RAG: External Memory Meets Generation
- 4 Why Vanilla RAG Is Not Enough
- 5 After RAG: Weaknesses and Fixes
- 6 Summary and Takeaways

Autoregressive generation: one token at a time

Each prediction depends on the input and the output so far

Example context c : “Business purchases have a 14-day refund window. What is the window?”

Step	Already generated $y_{<t}$	Next token y_t
1	(empty)	14
2	14	days
3	14 days	<eos>

Illustrative tokenization; <eos> ends the answer. θ : model parameters.

$$p_{\theta}(y \mid c) = \prod_{t=1}^T p_{\theta}(y_t \mid c, y_{<t}). \quad \mathcal{L} = - \sum_{t=1}^T \log p_{\theta}(y_t \mid c, y_{<t}).$$

Training: score the correct next token using the reference prefix.

Inference: choose a token, append it, then predict the next one.

Source: Vaswani et al. (2017), Section 3. Refund example: teaching illustration.

BERT, GPT, BART: different information paths

Encoding an input and generating an output are different jobs

Architecture	What can a position use?	What does it produce?
BERT Encoder-only	Input tokens on both sides: bidirectional self-attention.	Contextual token representations.
GPT Decoder-only	Earlier tokens, including the input prefix: causal self-attention.	A next-token distribution, repeatedly.
BART Encoder– decoder	Encoder: the input. Decoder: earlier output <i>and</i> encoder states.	A generated sequence conditioned on the encoded input.

Source: Lewis et al. (2020), BART, Section 2.1.

Attention: how output tokens use the input

Attention mixes information from positions available to the model

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}} + M\right) V.$$

Q, K, V : learned projections of token representations.

Queries Q match **keys** K ; the resulting weights mix **values** V .

M masks disallowed positions; a causal mask hides future output tokens.

Self-attention

Q, K, V come from the **same sequence**.

In a decoder-only LM, the prompt and generated prefix form that sequence. A new output position can attend to earlier prompt tokens.

Source: Vaswani et al. (2017), Sections 3.2.1–3.2.3; BART, Section 2.1.

Cross-attention

Q : decoder states.

K, V : encoder output states.

In BART, this gives the decoder access to the encoded input. Causal self-attention separately tracks the output prefix.

Fine-tuning vs. in-context conditioning

Changing the parameters is not the same as changing the input

Fine-tuning: change θ

$$\theta' = \theta - \alpha \nabla_{\theta} \mathcal{L}.$$

Training examples change the parameters through gradient updates.

Later predictions use the updated model $p_{\theta'}$. Those examples need not appear in the test prompt.

Few-shot ICL: change c

Translate English to French:

sea -> mer

cat -> chat

dog -> chien

Demonstrations are part of the prompt. The model conditions on them with **no parameter update**.

Two ways to give a model new information

Change θ : needs training; the knowledge stays fixed until the next update.

Change c : no training; but something must decide what goes into c .

Open question: if a needed fact is not in θ , how do we find the right text to put in c ?

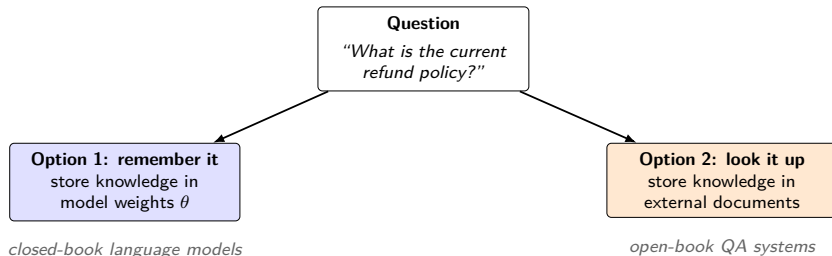
Source: Brown et al. (2020), Section 2. Prompt: teaching illustration; equation: an illustrative gradient step.

Table of Contents

- 1 LLM Background
- 2 Before RAG: Where Should Knowledge Live?
- 3 RAG: External Memory Meets Generation
- 4 Why Vanilla RAG Is Not Enough
- 5 After RAG: Weaknesses and Fixes
- 6 Summary and Takeaways

Where should the knowledge live?

Remember it in the weights, or look it up in documents?



Weights support fluent generation and broad learned knowledge.

Documents supply evidence that can be inspected and updated.

Source: Course material: *Before RAG*; Petko and Brody, previous lecture, "Two Ways to Answer."

But model weights are a poor database

Why “just memorize more” is not enough

Running example: a changing refund policy

The policy changes from **30 days** to **14 days**, but the model's weights stay fixed.

Limitation	Why it matters
Stale knowledge	The model may recall an outdated policy.
Missing knowledge	Private documents and rare facts may be absent from pretraining.
Difficult updates	Revising one fact is not a simple document edit.
Unclear provenance	A correct answer does not identify its supporting document.

Instead of storing every fact in the weights, let the model look up the evidence it needs.

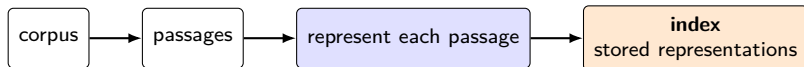
Source: *Before RAG* motivation; previous lecture, “Implicit Knowledge in Weights.” Policy example is illustrative.

Option 2: look up relevant documents

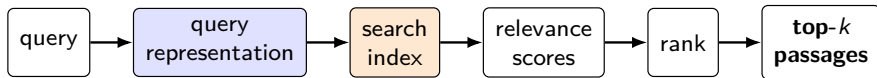
Retrieval selects a small set of passages from a large corpus

The model cannot read the whole corpus for every question. An **index** makes a small, relevant set accessible.

1. Offline: build a searchable index



2. Online: represent, search, score and rank



The retriever's output is ranked evidence, not the final answer.

Source: Course material: *Before RAG*, indexing; previous lecture, "Retrieve Before Generating" and "DPR Execution."

How do we score relevance? Sparse vs. dense

The same ranking problem, with different text representations

Sparse: term weights

TF-IDF / BM25

[0, 0, 2.1, 0, 1.3, 0, ...]

Most vocabulary-term weights are zero.

Match shared words through an inverted index.

Useful for exact terms; may miss paraphrases.

Dense: learned vectors

DPR

[0.12, -0.38, 0.71, 0.09, ...]

Learned embeddings; most entries are nonzero.

Compare query and passage vectors.

Similarity can match meaning beyond exact word overlap.

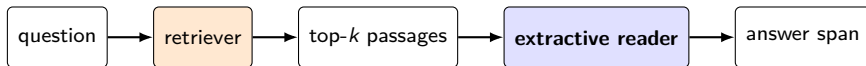
Recall the previous example: “bad guy” versus “villain.”

Different representations → relevance scores → top- k .

Source: Course review: Petko and Brody, BM25 and DPR; *Before RAG*, sparse/dense comparison.

The retriever finds passages; a reader must answer

Retrieve-then-read QA separates evidence search from answer extraction



Question: What is the refund window for the Business plan?

Rank	Retrieved passage
1	Business-plan purchases can be refunded within 14 days .
2	Business accounts include priority support.
3	Refund requests can be submitted through the billing portal.

An extractive reader selects an existing span: 14 days.

Source: *Before RAG* example and DrQA discussion (Chen et al., 2017).

Extraction is limited when the answer needs synthesis

The facts are present, but the requested explanation is not one span

Question: Compare the refund windows for Business and Student.

Retrieved passage A

Business-plan purchases can be refunded within **14 days**.

Retrieved passage B

Student-plan purchases can be refunded within **7 days**.

The answer we want

The Business plan has the **longer** refund window: **14 days**, compared with **7 days** for the Student plan.

Why an extractive reader cannot produce it:

- It uses facts from **two passages**; a span comes from one.
- It states a relation, “longer”, that **no passage contains**.

Synthesis = combining facts from several passages into new text. It needs a reader that **generates**, not one that copies.

Source: Illustrative Business/Student refund-policy example from the *Before RAG* course material.

Can retrieval learn which evidence helps?

REALM trains the retriever through the language-modeling loss

Learning relevance: DPR uses labeled query–passage pairs. Contriever learns from text spans without query labels.

REALM: learn retrieval through masked-token prediction

Example: $x = \text{“The [MASK] at the top of the pyramid”}$ $y = \text{“pyramidion”}$

$$p(y | x) = \sum_z \underbrace{p(z | x)}_{\text{retriever (BERT, inner product)}} \underbrace{p(y | x, z)}_{\text{reader (BERT reads } [x; z])} \quad \mathcal{L} = -\log p(y | x)$$

There is **one loss**: predict the masked token. No label says which z is correct.

Its gradient still reaches the retriever: if $p(y | x, z)$ is **above** the average $p(y | x)$, training **raises** $p(z | x)$; if below, it lowers it.

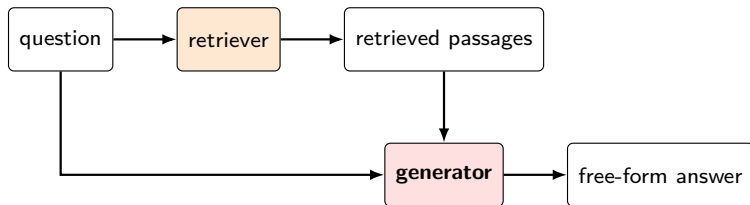
The next step: combine learned retrieval with a model that can generate a **complete output sequence**, rather than only predict a masked target or select an answer span.

Source: Guu et al. (2020), REALM, Figure 1 and Sections 2–3; Petko and Brody, previous lecture, “Training the Retriever,” Contriever, and “How REALM Works.”

Can we keep the strengths of both?

Evidence from retrieval, fluency from generation

	Closed-book LM	Retrieve-then-read
Writes a free-form answer (synthesis)	✓	× copies one span
Updatable, inspectable evidence	× fixed in θ	✓



Non-parametric memory: searchable, updatable documents.

Parametric memory: the generator's learned knowledge and language ability.

Source: Before RAG course material; Lewis et al. (2020), Section 1.

Table of Contents

- 1 LLM Background
- 2 Before RAG: Where Should Knowledge Live?
- 3 RAG: External Memory Meets Generation**
- 4 Why Vanilla RAG Is Not Enough
- 5 After RAG: Weaknesses and Fixes
- 6 Summary and Takeaways

Original RAG: what is the modeling contribution?

Lewis et al. (2020): Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks.

- Start from a **pre-trained retriever** and a **pre-trained generator**.
- Treat the retrieved passage as a **latent variable**.
- Fine-tune with input–output examples for different generation tasks.

Beyond the REALM review: the output need not be a masked token or an extracted answer span; it can be a free-form sequence.

Source: Lewis et al. (2020), Abstract and Sections 1–2.

Two distributions, two jobs

Component	Distribution	Question it answers
DPR retriever	$p_{\eta}(z \mid x)$	Which passage should help?
BART generator	$p_{\theta}(y_i \mid x, z, y_{<i})$	What should the next token be?

- The external Wikipedia index supplies the passage z .
- BART supplies the ability to generate a sequence y .
- The model combines predictions across the retrieved passages.

Source: Lewis et al. (2020), Sections 2.1–2.3.

From DPR scores to document probabilities

Recall the previous lecture's inner-product score:

$$s(x, z) = q(x)^\top d(z).$$

Within the retrieved top- K set $\mathcal{Z}_K(x)$, normalize the scores:

$$p_\eta(z \mid x) = \frac{\exp s(x, z)}{\sum_{z' \in \mathcal{Z}_K(x)} \exp s(x, z')}.$$

	Passage (x : "What is the Business plan's refund window?")	$s(x, z)$	$p_\eta(z \mid x)$
z_1	Business-plan purchases can be refunded within 14 days.	71.2	0.70
z_2	Business accounts include priority support.	70.35	0.30

Illustrative scores, $K = 2$.

Why probabilities? RAG does not know which passage is right, so it averages over the passages. The weights of that average must be **positive and sum to 1**: softmax turns 71.2 and 70.35 into **0.70** and **0.30**, and the higher score keeps the larger weight.

Source: Lewis et al. (2020), Section 2.2. Scores: teaching illustration.

What does BART actually read?

BART reads the passage and the question **concatenated**, once per retrieved passage.

	BART input $[z; x]$	Output
Run 1	Business-plan purchases can be refunded within 14 days. What is the Business plan's refund window?	$p_{\theta}(y \mid x, z_1)$
Run 2	Business accounts include priority support. What is the Business plan's refund window?	$p_{\theta}(y \mid x, z_2)$

Orange: the retrieved passage z . Black: the question x .

Two separate inputs, two separate answer probabilities.

They are combined with the weights **0.70** and **0.30** from the previous slide.

Source: Lewis et al. (2020), Section 2.3; refund text is a teaching illustration.

Combine the two runs: average over the passage

Two runs give two answer probabilities. What is the **one** probability $p(y | x)$?

	weight $p_{\eta}(z x)$	answer prob. $p_{\theta}(y x, z)$	product
Run 1, passage z_1	0.7	0.9	0.63
Run 2, passage z_2	0.3	0.2	0.06

$$p(y | x) \approx \sum_{z \in \mathcal{Z}_K(x)} p_{\eta}(z | x) p_{\theta}(y | x, z) = 0.63 + 0.06 = 0.69.$$

Why average instead of just using z_1 ?

- The training data has only (x, y) ; nothing says which passage is right.
- The average depends on the weights, so training can adjust them: moving to 0.9 / 0.1 raises $p(y | x)$ from 0.69 to $0.9 \times 0.9 + 0.1 \times 0.2 = 0.83$.

Summing out the unknown passage is called **marginalization**; z is a **latent variable**.

Source: Lewis et al. (2020), Section 2; numbers are illustrative, not experimental results.

RAG-Sequence: one passage for the whole answer

Question: “What does the Business plan include?” (needs **14**-day refunds *and* **priority** support)

$$p_{\text{Seq}}(y \mid x) \approx \sum_z p_{\eta}(z \mid x) \underbrace{\prod_i p_{\theta}(y_i \mid x, z, y_{<i})}_{\text{whole answer under one } z}$$

	weight	$p(\text{"14"})$	$p(\text{"priority"})$	row product
z_1 : refund passage	0.7	0.9	0.1	0.09
z_2 : support passage	0.3	0.1	0.9	0.09

$$p_{\text{Seq}} = 0.7 \times 0.09 + 0.3 \times 0.09 = \mathbf{0.09}$$

Each passage must explain the **whole** answer alone.

Source: Lewis et al. (2020), Section 2.1; numbers are illustrative.

RAG-Token: a different passage for each token

Same question, passages and weights.

$$p_{\text{Tok}}(y \mid x) \approx \prod_i \underbrace{\sum_z p_{\eta}(z \mid x) p_{\theta}(y_i \mid x, z, y_{<i})}_{\text{mix the passages for each token}}$$

	weight	$p(\text{"14"})$	$p(\text{"priority"})$
z_1 : refund passage	0.7	0.9	0.1
z_2 : support passage	0.3	0.1	0.9
weighted average		0.66	0.34

$$p_{\text{Tok}} = 0.66 \times 0.34 \approx \mathbf{0.22} \quad (\text{vs. } 0.09)$$

Each token can lean on a **different** passage.

Source: Lewis et al. (2020), Section 2.1; numbers are illustrative.

Training: answer supervision reaches the retriever

$$\mathcal{L}(\eta, \theta) = - \sum_j \log p(y^{(j)} | x^{(j)}).$$

Component	During fine-tuning	Why
Query encoder	Updated	Learns which passages help the output
BART generator	Updated	Learns to use retrieved text
Document encoder / index	Fixed	Avoids repeatedly re-encoding the corpus

RAG fine-tuning needs no gold-document labels. The starting DPR retriever, however, was trained with gold passages on Natural Questions and TriviaQA.

Source: Lewis et al. (2020), Sections 2.2 and 2.4.

Evidence: open-domain QA results

Question: can retrieval plus generation compete with an extractive reader?

	NQ	TriviaQA	WebQuestions	CuratedTREC
Questions from	Google search	trivia quizzes	Google Suggest	TREC QA track
Test questions	3,611	11,314	2,033	635
DPR (extractive)	41.5	57.9	41.1	50.6
RAG-Token	44.1	55.2	45.5	50.0
RAG-Sequence	44.5	56.8	45.2	52.2

Exact match (%), test sets. DPR = retriever + re-ranker + extractive reader.

- **Generation vs. extraction:** RAG beats DPR on NQ and WebQuestions without a re-ranker; DPR still wins TriviaQA.
- **Why generation helps:** on NQ questions where no retrieved passage contains the answer, RAG still gets 11.8%; an extractive reader gets 0%.
- **Token vs. Sequence:** gaps are small and mixed; 0.3 points on WebQuestions is about 6 questions.

Source: Lewis et al. (2020), Table 1, Section 4.1 and Appendix (dataset sizes).

Evidence: retrieval makes generation more factual

Same generator, with and without retrieval: BART alone vs. RAG.

MS-MARCO (answer with a full sentence). Input: *define middle ear*

BART	The middle ear is the part of the ear between the middle ear and the nose.	×
RAG-Seq	The middle ear includes the tympanic cavity and the three ossicles.	✓

Jeopardy question generation (write a clue for a given answer). Raters compared 452 BART/RAG pairs:

	More factual	More specific
BART better	7.1%	16.8%
RAG better	42.7%	37.4%

Remaining pairs: both good, both poor, or no majority.

Takeaway: BART alone invents facts; with retrieved text, raters judged RAG more factual 6× as often.

Source: Lewis et al. (2020), Tables 3–4.

Ablation: does the retriever need to be learned?

Same RAG-Sequence model; only the retriever changes.

Retriever	What it is	NQ	TriviaQA	FEVER-3
BM25	word overlap, no learning	31.8	44.1	75.1
DPR, frozen	dense, not updated by RAG	41.2	52.1	72.9
DPR, fine-tuned	dense, query encoder updated	44.0	55.8	74.5

NQ, TriviaQA: exact match. FEVER-3: label a claim as supported, refuted, or not enough info. Dev sets.

- **Dense beats word overlap** on QA: +9.4 on NQ, +8.0 on TriviaQA.
- **Learning helps further:** fine-tuning adds +2.8 and +3.7.
- **Exception, FEVER:** claims name specific entities, so word overlap already finds the right page.

Source: Lewis et al. (2020), Table 6 and Section 4.5.

Can knowledge change without retraining BART?

Setup: 82 world leaders who changed between 2016 and 2018. Ask the NQ RAG model “Who is {position}?” (e.g. “Who is the President of Peru?”). **Only the Wikipedia index is swapped.**

Index built from	Answers the 2016 leader	Answers the 2018 leader
Wikipedia, Dec. 2016	70%	4%
Wikipedia, Dec. 2018	12%	68%

- **Matched index:** about 70% correct for that year.
- **Mismatched index:** only 4–12%. The answer follows the index, not what BART memorized.

Design benefit: to update knowledge, rebuild the index; BART stays fixed. A parametric-only model would need more training.

Source: Lewis et al. (2020), Section 4.5 (index hot-swapping).

Original RAG is not just passage concatenation

	Original RAG model	Common assembled RAG pattern
Generator input	Query plus one passage per conditional branch	Query plus selected passages in a prompt
Combination	Marginalize over latent passages	Generate from the assembled context
Training	Joint RAG fine-tuning	Often use separately trained components
Key dependency	Useful document-conditioned predictions	Good retrieval, context construction and evidence use

Next: accessing evidence does not guarantee a complete, well-supported answer.

Source: Lewis et al. (2020), Section 2; Barnett et al. (CAIN 2024), Section 3. Comparison is a teaching synthesis.

Table of Contents

- 1 LLM Background
- 2 Before RAG: Where Should Knowledge Live?
- 3 RAG: External Memory Meets Generation
- 4 Why Vanilla RAG Is Not Enough**
- 5 After RAG: Weaknesses and Fixes
- 6 Summary and Takeaways

Production RAG has several places to lose an answer

Barnett et al. study **assembled RAG systems**, not a re-run of Lewis et al.'s jointly fine-tuned model.

Stage	What must go right
Ingestion	The needed content is present and processed correctly.
Retrieval	The right evidence makes the candidate list.
Context assembly	The evidence survives the token budget.
Generation	The reader uses it, answers fully and follows the requested format.

Source: Barnett et al. (CAIN 2024), Figure 1 and Section 3; plain-text reading of the reported pipeline.

The paper's seven failure points

FP	Name	What goes wrong
1	Missing content	The answer is absent from the accessible documents.
2	Missed top ranked	The right passage is below the retrieval cutoff.
3	Not in context	Retrieved evidence is dropped before reaching the reader.
4	Not extracted	The reader misses information that is in its context.
5	Wrong format	The requested output structure is not followed.
6	Incorrect specificity	The answer is too broad or too detailed.
7	Incomplete	Part of the available answer is omitted.

Do not diagnose every error as a retrieval error. FP4–FP7 can happen even after the right text reaches the reader.

Source: Barnett et al. (CAIN 2024), Section 5.

Three case studies in three domains

System	Documents	Size	Sample question
Cognitive Reviewer	Research PDFs	Variable	What are the key points across these papers?
AI Tutor	Videos, HTML, PDF	38	What topics were covered in week 6?
BioASQ	Scientific documents	4,017	A biomedical question with an expert answer

Research, education and biomedicine expose different requirements for **coverage, context and answer quality**.

The first two systems were used at Deakin University; the paper's public experimental case is BioASQ.

Source: Barnett et al. (CAIN 2024), Section 4 and Table 1.

Case study 1: Cognitive Reviewer

C | Helping PhD students review the literature

What it does

- A researcher states a research question, then uploads a set of papers.
- The system **ranks** all papers by relevance to that question for manual review, and answers questions asked against all of them.
- Stages: chunker, rewriter, retriever, reader.

Engineering challenge: indexing at run time

Indexing (splitting documents into chunks and turning them into searchable vectors) happens **when the user uploads**, not in advance. So engineers **cannot quality-check the documents** during development; everything depends on a robust data-processing pipeline.

Failure point in practice: FP7 incomplete

“What are the key points covered in documents A, B and C?” tends to miss information. Asking about each document **separately** works better.

Source: Barnett et al., Sections 4.1 and 5, Table 1.

Case study 2: AI Tutor

C | Answering students' questions from course material

What it does

- Built into Deakin's learning management system; indexes PDFs, text, and **videos**, which are first turned into text by **Whisper** (a speech-to-text model).
- Students see a **sources list** to verify each answer.
- Built Aug.–Nov. 2023; piloted with **200 students** from 30 Oct. 2023.

The rewriter

A **rewriter** uses an LLM to turn the chat history plus a vague request (“Explain this concept further”) into a clear, stand-alone search query.

Observed

- A larger **context window** (how much text the LLM can read at once; 8K vs. 4K tokens) gave more accurate answers.
- Adding **file name and chunk number** to retrieved text helped the reader.
- FP6: a teacher wants **educational content** with the answer, not just the bare answer.

Source: Barnett et al., Sections 4.2 and 5, Table 2.

Case study 3: Biomedical QA (BioASQ)

C | Testing failure points at a larger scale

Setup

- **BioASQ**: a dataset of questions, linked documents, and answers written by **biomedical experts**.
- Question types: yes/no, text summary, **factoid** (one short fact, like a name or number), and list.
- 4017 open-access documents indexed; **1000 questions** asked; answers generated by **GPT-4**.
- Stages: chunker, retriever, reader (no rewriter).

Why this case matters

The first two systems had **small** document sets. BioASQ checks whether the same failures appear at scale, and it is the only case whose scripts and data were **released publicly**.

Why this mattered → Three domains produced the same kinds of breakdowns, so the next question is how they were measured and classified.

Source: Barnett et al., Section 4.3, Table 1.

How were the failures evaluated?

- 1 Use automatic evaluation to flag potentially incorrect answers.
- 2 Inspect flagged answers and a sample of the others manually.
- 3 Classify where the answer failed in the pipeline.

Important limits in the report: the automatic evaluator was more pessimistic than human raters; those raters were not biomedical experts.

The paper reports 4,017 documents in the BioASQ setup (Section 4.3), although its introduction gives a different corpus-size description.

Source: Barnett et al. (CAIN 2024), Sections 1, 4.3 and 5.

Debug the component that lost the evidence

What to inspect	Diagnostic question	Relevant FPs
Corpus / ingestion	Does the answer exist in the index?	1
Retriever / reranker	Did the passage enter the top- K ?	2
Consolidator	Did it survive context trimming?	3
Reader prompt / model	Was the evidence used correctly?	4–7

For example: raising K cannot create a missing source document, and a better embedding cannot by itself fix an output-format error.

Source: Barnett et al. (CAIN 2024), Sections 3 and 5; diagnostic questions are teaching synthesis.

The case studies motivate different repairs

- **Incomplete coverage:** how can the system gather evidence across many documents?
- **Distracting evidence:** how can the reader decide whether a passage helps?
- **Weak context:** how can chunk metadata and surrounding meaning be preserved?

The transition to After RAG

Do not choose a method just because its name contains “RAG.” First identify the failure it is meant to address.

Source: Barnett et al. (CAIN 2024), Table 2 and Section 6; links to later methods are teaching synthesis.

Table of Contents

- 1 LLM Background
- 2 Before RAG: Where Should Knowledge Live?
- 3 RAG: External Memory Meets Generation
- 4 Why Vanilla RAG Is Not Enough
- 5 After RAG: Weaknesses and Fixes**
- 6 Summary and Takeaways

What vanilla RAG leaves open

Weakness of vanilla RAG	Idea	Method
Always retrieves K passages; never checks whether the answer is supported.	Let the model decide when to retrieve and critique its own output.	Self-RAG
Top- K similar chunks cannot answer corpus-wide questions ("What are the main themes?").	Summarize communities of a knowledge graph, then aggregate.	GraphRAG
Each passage is matched to the query separately, so evidence linked only through an entity is missed.	Store entity links in a graph; spread relevance along it.	HippoRAG
Relevance is one inner product against an external vector index.	Let the model generate the identifier of the evidence.	DSI / SEAL

Source: Teaching synthesis of the method papers cited on the following slides.

Self-RAG: first read the paper's example

Retrieval-Augmented Generation (RAG)

Prompt How did US states get their names?

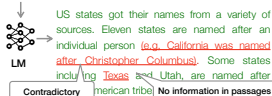
Step 1: Retrieve K documents

- 1 Of the fifty states, eleven are named after an individual person.
- 2 Popular names by states. In Texas, Emma is a popular baby name.
- 3 California was named after a fictional island in a Spanish book.

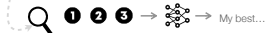
Retriever

Step 2: Prompt LM with K docs and generate

Prompt How did US states get their names? + 1 2 3



Prompt: Write an essay of your best summer vacation



Ours: Self-reflective Retrieval-Augmented Generation (Self-RAG)

Prompt How did US states get their names?

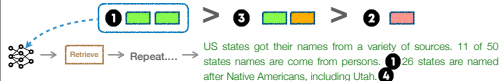
Step 1: Retrieve on demand



Step 2: Generate segment in parallel



Step 3: Critique outputs and select best segment



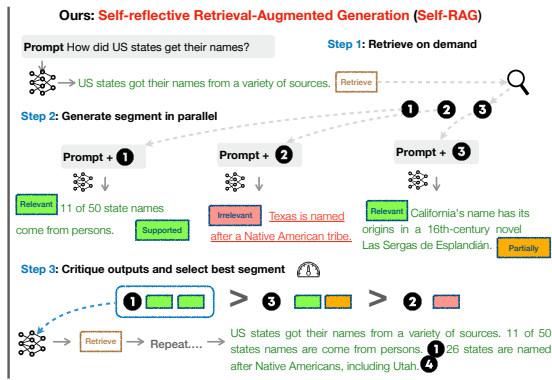
Prompt: Write an essay of your best summer vacation



Left: retrieved passages can still produce contradictions.
Right: decide when to retrieve, then evaluate candidate segments.

Source: Asai et al. (2023; ICLR 2024), Figure 1.

Self-RAG: enlarge the decision-and-critique steps



1. Retrieve

Only when useful.

2. Generate

One candidate per passage.

3. Critique

Prefer relevant, supported output.

The chosen segment extends the answer; the process can continue.

Source: Asai et al. (2023; ICLR 2024), Figure 1, right-panel detail (enlarged crop).

Self-RAG: reflection is learned, not just requested

Token type	Decision / assessment
Retrieve	Retrieve now, skip, or continue with current evidence?
IsRel	Does this passage help answer the question?
IsSup	Are the segment's claims supported by the passage?
IsUse	Is the response useful for the requested task?

Training: teacher judgments train a critic; the critic annotates data offline; the generator learns ordinary text and reflection tokens together.

At inference: the generator produces its own reflection tokens. A predicted support label is still a model judgment, not proof.

Source: Asai et al. (2023; ICLR 2024), Table 1 and Sections 3.2–3.3.

Self-RAG on ASQA: what improved?

ASQA: ambiguous questions, e.g. “When did *The Simpsons* first air?” (1987 as shorts; 1989 as a series). The answer must cover every reading and cite retrieved passages.

Metric	What it measures	FT 7B	Self-RAG 7B
str-em	Gold short answers in the output	31.0	30.0
ROUGE-L	Word overlap with the reference	35.8	35.7
MAUVE	Fluency (how human-like it reads)	51.2	74.3
Citation precision	Citations that support their claim	5.0	66.9
Citation recall	Claims supported by their citations	7.5	67.8

FT 7B: Llama2-7B fine-tuned on the same data *without* reflection tokens; both read retrieved passages at test time.

Read the contrast: reflection tokens make answers **verifiable** and more fluent, but **not more correct** (str-em 31.0 vs. 30.0).

Source: Asai et al. (2023), Table 2 and Section 4; ASQA: Stelmakh et al. (2022); citation metrics: ALCE, Gao et al. (2023).

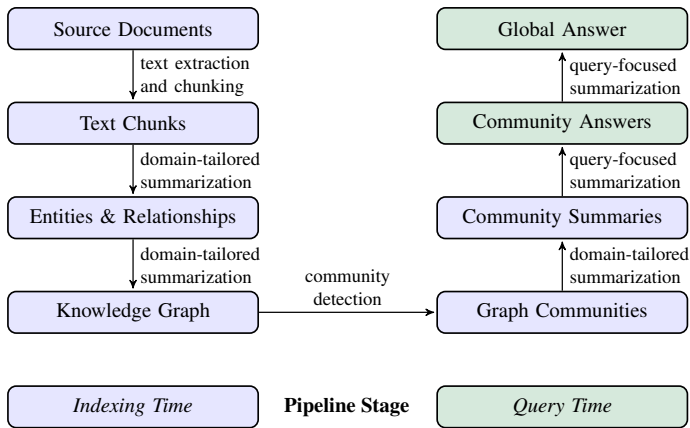
GraphRAG: a global question is not a local lookup

Local question	Global question
What is the refund deadline for this plan?	What themes recur across the support-ticket corpus?
A small set of directly relevant passages may be enough.	A few similar passages may omit entire topics.

Core idea: organize and summarize the corpus before answering global questions.

Source: Edge et al. (2024; v2), Sections 1–2; support-ticket questions are illustrative.

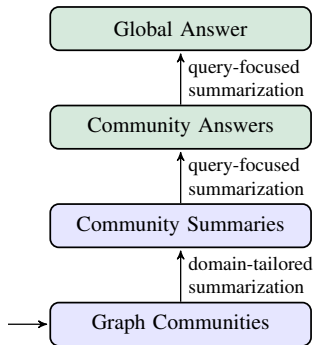
GraphRAG: the original indexing-and-query figure



Follow the purple indexing stages first, then the green query stages.

Source: Edge et al. (2024; v2), Figure 1.

GraphRAG: why communities change the context



Before the query

Detect communities and write summaries.

At query time

Get partial answers from summaries, then aggregate them.

What changes?

Context represents corpus regions, not only similar chunks.

Trade-off: graph extraction and summaries cost work and can lose details.

Source: Edge et al. (2024; v2), Figure 1, right-side detail; Section 2.

GraphRAG results: which answer does the judge prefer?

Setup: podcast transcripts ($\approx 1\text{M}$ tokens) and news articles ($\approx 1.7\text{M}$ tokens); 125 global questions each, e.g. “Which episodes deal primarily with tech policy?” There are **no gold answers**, so an LLM judge compares GraphRAG (C3, finest community level) with vector RAG, head to head.

Criterion	The judge asks	Podcasts	News
Comprehensiveness	Does it cover every aspect?	79.0	79.4
Diversity	Does it give varied perspectives?	80.8	69.1
Empowerment	Does it help the reader judge?	49.0	49.5
Directness	Does it answer specifically and clearly?	40.5	45.6

GraphRAG's win rate (%) over vector RAG; a tie counts as half a win; five judging runs.

Read it: GraphRAG answers are **broad**er (wins on the first two) but **less direct** (loses the last). These are judge preferences, not accuracy.

Source: Edge et al. (2024; v2), evaluation criteria and Table 6 (C3 vs. S5).

HippoRAG: the missing passage may be one hop away

Question: What is Alice's refund window?

Evidence	Text
----------	------

Passage A	Alice is subscribed to the Business Plan .
-----------	---

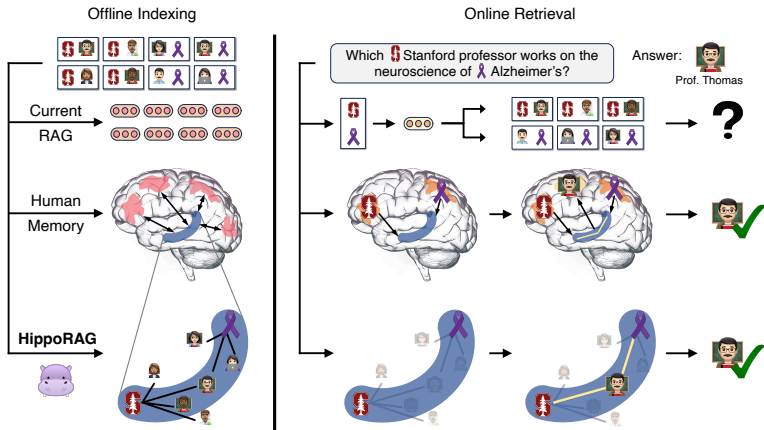
Passage B	The Business Plan has a 14-day refund window .
-----------	--

The query names Alice; Passage B does not. Direct similarity can miss the second piece of evidence.

Bridge: Business Plan connects the two passages. Graph-based association can help recover that connection.

Source: Gutiérrez et al. (2024), motivation in Sections 1–2. Alice is a teaching illustration, not a reported benchmark case.

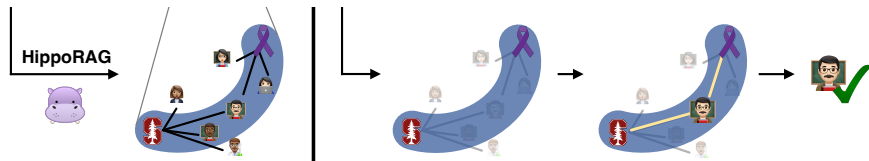
HippoRAG: compare the three rows in the paper



The bottom row retrieves through associations in a stored graph.

Source: Gutiérrez et al. (2024), Figure 1.

HippoRAG: how association becomes retrieval



- 1 Extract phrase relations and build the memory graph offline.
- 2 Link query entities to graph seeds; run **Personalized PageRank**.
- 3 Use graph relevance to rank and retrieve the supporting passages.

Versus GraphRAG global search: retrieve connected evidence, rather than aggregate community-summary answers.

Source: Gutiérrez et al. (2024), Figure 1, bottom-row detail; Section 2. GraphRAG comparison is teaching synthesis.

HippoRAG: gains depend on how the questions are built

Same dense encoder (Contriever); only the retrieval structure changes.

Dataset	What the questions are like	Dense	HippoRAG	Gain
MuSiQue	2–4 hops, composed so that one-passage shortcuts fail	46.6	52.1	+5.5
2Wiki	2 hops from templates over entities (“Where was <i>the director of film X</i> born?”)	57.5	89.5	+32.0
HotpotQA	2 hops written by crowdworkers; one passage often suffices	75.5	76.2	+0.7

Recall@5 (%): share of the needed passages found in the top 5. 1,000 dev questions per dataset.

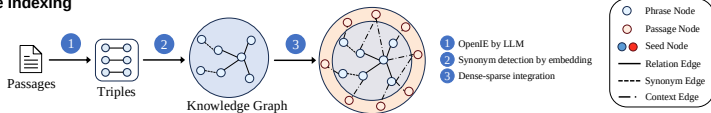
Read it: the gain is largest when answering requires **linking entities** across passages (2Wiki), and nearly zero when one passage often suffices (HotpotQA).

Recall measures finding the passages, not answering the question.

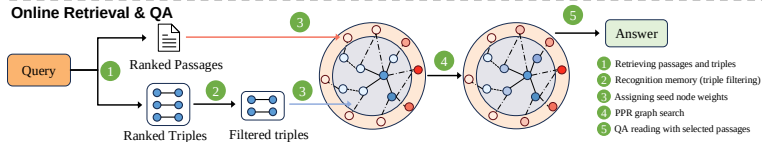
Source: Gutiérrez et al. (2024), Table 2, Datasets section and Appendix (dataset comparison).

HippoRAG 2: put passage context back into memory

Offline Indexing



Online Retrieval & QA



Top row: integrate passage nodes with phrase-level structure.

Bottom row: retrieve passages and triples, filter triples, seed the graph, run PPR, then answer from selected passages.

Source: Gutiérrez et al. (2025), Figure 2.

HippoRAG 2: keep the multi-hop gain, fix the losses

All three share the embedder (NV-Embed-v2) and reader (Llama-3.3-70B).

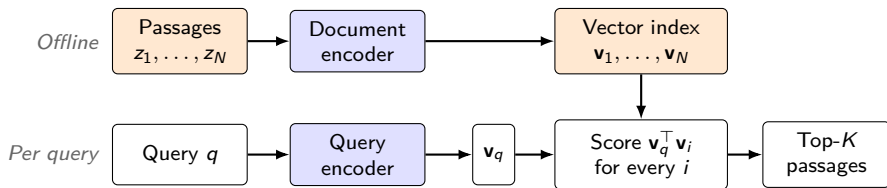
Type	Dataset	Dense	HippoRAG	HippoRAG 2
Simple QA (one fact)	NQ (real user questions)	61.9	55.3	63.3
	PopQA (less popular entities)	55.7	55.9	56.2
Multi-hop QA (link facts)	MuSiQue	45.7	35.1	48.6
	2Wiki	61.5	71.8	71.0
	HotpotQA	75.3	63.5	75.5

F1: word overlap with the gold answer. Red: below dense retrieval. 1,000 questions per dataset.

- **HippoRAG:** +10.3 on entity-centric 2Wiki, but −6.6 on NQ, −10.6 on MuSiQue, −11.8 on HotpotQA.
- **HippoRAG 2:** adds passage nodes back; never below dense, and keeps most of the 2Wiki gain.

Source: Gutiérrez et al. (2025), Table 2 and Datasets section.

Recap: dense retrieval searches stored vectors

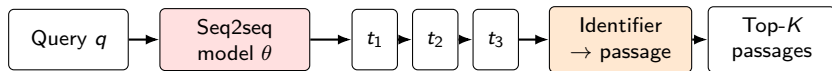


- **Relevance** is one inner product between two vectors that were computed *separately*.
- **The corpus lives outside the model**, as N stored vectors (RAG: about 21M; the document encoder stays frozen).
- **Retrieval is a search** over those vectors (maximum inner-product search), not an output of the network.

Question: could one model read q and directly *output* where the evidence is?

Source: Karpukhin et al. (2020), DPR; Lewis et al. (2020), Section 2.4; Tay et al. (2022), Table 1.

Generative retrieval: decode where the evidence is



*identifier, one token at a time;
beam search keeps the K best*

	Dense retrieval	Generative retrieval
Relevance score	$\mathbf{v}_q^\top \mathbf{v}_z$	$P(\text{id} \mid q) = \prod_t P(\text{id}_t \mid q, \text{id}_{<t})$
Top-K comes from	Nearest-vector search	Beam search over identifiers
Corpus lives in	External vector index	Model parameters, plus an ID or text lookup

Same decoding as slide 4, but the output is an *identifier*.

DSI: a document ID, e.g. 3 7 2

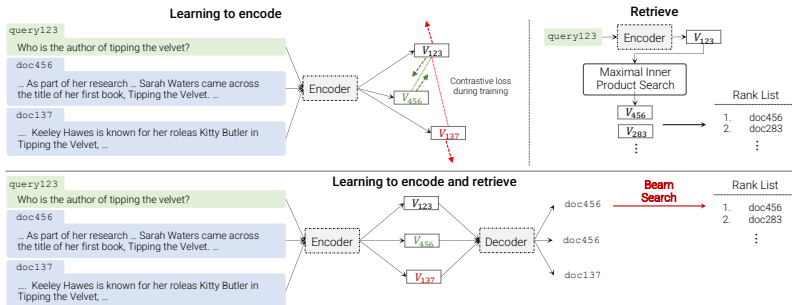
SEAL: a string from the passage, e.g. “carbon dioxide emissions”

It generates a pointer to evidence, not the answer; a reader still answers.

Source: Tay et al. (2022), Section 3; Bevilacqua et al. (2022), Section 4 and Figure 1. IDs are illustrative.

DSI: the model memorizes the index

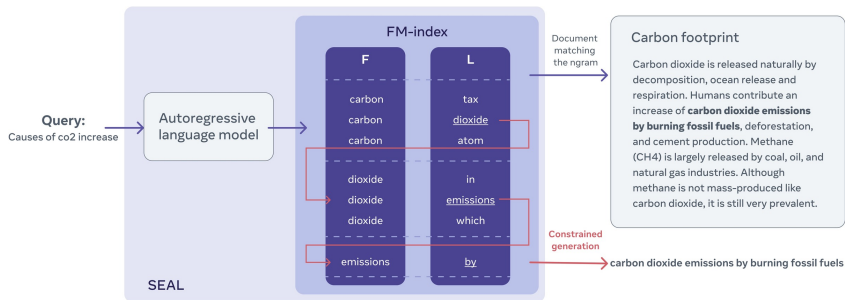
- Give each document an ID; best: a **semantic ID** from clustering, so similar documents share prefixes.
- Train one T5 on **document text** $\rightarrow$ **ID** and **query** $\rightarrow$ **ID**.
- Beam search decodes IDs; the corpus now lives in the parameters θ .



Top: dual encoder (encode, then inner-product search). Bottom: DSI (decode the ID).

SEAL: the corpus's own words are the identifiers

- No IDs to assign: any string that occurs in a passage identifies it.
- An **FM-index** (compressed full-text index) lets BART generate only strings found in the corpus, then returns the passages containing them.
- Passages are ranked by the strings they contain, favoring strings that are likely given the query but rare in the corpus.



Red path: each next word must continue a string found in the corpus.

Limitations: why dense retrieval is still the default

	DSI	SEAL
Scale (21M passages)	Acc@5 28.3 vs. DPR 68.3	Acc@5 61.3 vs. DPR 68.3; ties at Acc@100
New documents	New IDs and more training	Rebuild the text index only
Stability	Forgets during training: Hits@1 40.4 (best) vs. 12.2 (worst checkpoint)	—
Query cost	Seq2seq beam search per query	BART-large beam search per query

Acc@ k : a top- k passage contains the answer. DSI at 21M passages is the SEAL authors' BART replication; Hits@1 is on NQ320K.

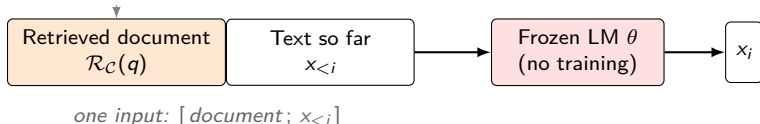
Contrast with slide 30: RAG updates knowledge by swapping the index; DSI cannot.

What survived: semantic IDs (generative recommendation) and corpus-constrained decoding (generated evidence is guaranteed to exist).

Source: Bevilacqua et al. (2022), Table 3 and Section 5; Tay et al. (2022), Tables 2 and 6.

In-Context RALM: prepend, keep the LM frozen

*BM25 query = last $\ell = 32$ tokens;
retrieve again every $s = 4$ tokens*



$$p(x_1, \dots, x_n) = \prod_{i=1}^n p_{\theta}(x_i \mid [\mathcal{R}_C(q); x_{<i}])$$

- **No change to the LM:** same weights, same architecture; works even through an API.
- **Off-the-shelf retriever:** for language modeling, BM25 beat the dense retrievers tested.

Evidence: GPT-2 345M + BM25 reaches 21.5 perplexity on WikiText-103, better than GPT-2 762M without retrieval (22.0). With the top-2 DPR passages in the prompt, LLaMA-13B rises from 12.0 to 31.0 exact match on NQ.

Source: Ram et al. (2023), In-Context Retrieval-Augmented Language Models, TACL, Section 3, Tables 1 and 4.

Same idea, different model

“RAG” now names a **pattern**, not the 2020 **model**.

	Lewis et al. (2020)	In-Context RALM (2023)	ChatGPT / Claude today
When to retrieve	Once, from the input	Every s tokens, from the recent text	The model decides; it may search again
Where from	A fixed Wikipedia index	A fixed corpus	The web, your files, your past chats
How text is used	One passage per branch, then marginalize	Prepended to the input	Top- K results placed in the prompt
Training	Retriever and generator fine-tuned jointly	None: frozen LM, off-the-shelf retriever	Built separately; the LLM is trained to use tools
Output	An answer	The next tokens	An answer with citations

Unchanged: generation is conditioned on retrieved text, $p_{\theta}(y \mid x, z)$.

Changed: no marginalization, no joint training (per public docs); retrieval is a tool call, the “assembled” pattern of slide 32.

Source: Lewis et al. (2020), Sec. 2; Ram et al. (2023), Sec. 3; OpenAI: ChatGPT search, File search; Anthropic: Web search tool, RAG for projects, Chat search (accessed Sep. 27, 2026). Teaching synthesis.

Table of Contents

- 1 LLM Background
- 2 Before RAG: Where Should Knowledge Live?
- 3 RAG: External Memory Meets Generation
- 4 Why Vanilla RAG Is Not Enough
- 5 After RAG: Weaknesses and Fixes
- 6 Summary and Takeaways

Takeaways

Stage	Key idea
Before RAG	Weights generate fluently but are fixed; retrieve-then-read has evidence but only copies a span.
RAG (2020)	Retriever + generator. The passage is a latent variable, marginalized; answers alone train the retriever. Swap the index to update knowledge.
After RAG	Each method fixes one weakness: Self-RAG (when to retrieve, is it supported), GraphRAG (corpus-wide questions), HippoRAG (multi-hop links). DSI/SEAL make the model the index, but struggle to scale and update.
Today	“RAG” is a pattern: search as a tool, retrieved text placed in the prompt (In-Context RALM); no joint training or marginalization.

One thread through every method: deciding what text the model should read before it answers.

Source: Lecture synthesis of the papers cited on the preceding slides.